

EDUGENIE

Google Gemini Powered Learning Assistant

GROUP PROJECT DOCUMENTATION

MAHARANI ARTS AND SCIENCE COLLEGE

BHARATHIYAR UNIVERSITY

PROJECT TEAM

Mathina Begam M — Team Lead

Ishwarya K — Member

Monisha S — Member

Leelamery A — Member

Sofiya Parveen A — Member

Documentation Basis and Implementation Verification

Implementation verification note: the supplied original EduGenie PDF was used as the structural and visual reference. The implementation-specific content in this document is aligned to the current public EduGenie-AI repository visible at github.com/mathiina/EduGenie-AI. Where the older PDF describes Gemini or a local model for a feature that the current source implements locally, this document follows the source code rather than repeating the older claim.

The original supplied PDF presents EduGenie as a lightweight educational assistant with question answering, concept explanation, quiz generation, summarization, and learning recommendations. The current repository README confirms the same five user-facing capabilities and lists Python, FastAPI, HTML, CSS, JavaScript, Jinja2, Google Gemini API, and LaMini-Flan-T5 among the project technologies.

For implementation accuracy, the current source code is treated as authoritative. In the current code, Q&A and quiz generation call Gemini; explanation is a local template-based function; summary keeps the first three sentence fragments; and learning recommendations are local rule-based lists selected by the requested level. The README still mentions LaMini-Flan-T5, but the current `explanation_module.py` does not load or invoke that model.

Contents

1. Project Overview
2. Problem Statement
3. Proposed Solution
4. Objectives
5. Main Use Cases
6. Technologies and Requirements
7. Prerequisites
8. System Architecture
9. Request Flow
10. Project Folder Architecture
11. Milestone 1 – Model Selection and Architecture
12. Milestone 2 – Backend Module Development
13. Milestone 3 – Frontend Development
14. Milestone 4 – Running and Testing
15. Project Evidence
16. Git and GitHub Workflow
17. Repository Safety
18. Advantages
19. Limitations
20. Future Enhancements
21. Conclusion
22. References
23. Submission Details

1. Project Overview

EduGenie is a lightweight web-based learning assistant that brings several common study activities into one interface. A user selects a learning module, provides a question, topic, or passage, and receives a generated or locally produced result in the same page. The project uses FastAPI as the backend framework, Jinja2 for serving the HTML template, static CSS and JavaScript for the browser interface, and a Gemini client for the AI-dependent modules.

The current repository contains separate Python modules for Q&A, explanation, quiz generation, summarization, and learning recommendations. This modular structure keeps the API layer in `main.py` relatively small and allows individual learning functions to be developed independently.

2. Problem Statement

Students often move between different tools when they need a direct answer, a simpler explanation, a quick quiz, a short revision summary, or a suggested learning sequence. This can interrupt the learning process and make self-study less organized. EduGenie addresses this problem by placing these study operations behind a single browser interface and a common FastAPI backend.

3. Proposed Solution

The proposed solution is a modular educational assistant in which the frontend captures the user's selected task and input, JavaScript converts the selection into the appropriate POST request, FastAPI dispatches the request to the matching Python function, and the result is returned to the browser. Gemini is used where the current code requires generative AI, while simpler deterministic functions are used for explanation, summary, and learning-path recommendations.

4. Objectives

- Provide a single interface for common learning-support tasks.
- Keep the backend modular by separating each educational capability into its own Python file.
- Use Gemini for Q&A and structured quiz generation.
- Return quiz data in a predictable JSON structure and validate the generated questions.
- Provide a simple, responsive browser interface using HTML, CSS, and JavaScript.
- Keep API credentials outside the source code through environment variables.

5. Main Use Cases

Use Case	User Input	System Output
Q&A	Question	Gemini-generated answer
Explanation	Concept / topic	Simple local explanation
Quiz	Topic or passage	3 MCQs with 4 options each
Summary	Text / passage	First three sentence fragments,

6. Technologies and Requirements

- Python – application language.
- FastAPI – backend web framework and REST endpoint layer.
- Uvicorn – ASGI server used to run the application.
- Jinja2 – HTML templating used by the FastAPI frontend route.
- HTML, CSS, JavaScript – browser interface.
- Google GenAI SDK – Gemini client used by Gemini-dependent modules.
- Pydantic – request validation models in main.py.
- python-dotenv – loads Gemini configuration from environment variables.
- pytest and httpx – included in the repository requirements for testing support.
- Transformers, Torch, and SentencePiece – present in requirements.txt, although the current visible explanation module does not invoke a transformer model.

Repository Dependency List

- fastapi>=0.115.0
- uvicorn[standard]>=0.30.0
- jinja2>=3.1.4
- python-dotenv>=1.0.1
- python-multipart>=0.0.9
- google-genai>=1.30.0
- pydantic>=2.8.0
- transformers>=4.45.0
- torch>=2.2.0
- sentencepiece>=0.2.0
- pytest>=8.0.0
- httpx>=0.27.0

7. Prerequisites

- Python 3.10 or later.
- pip and a working Python virtual environment.
- A browser such as Chrome, Edge, or Firefox.
- A Gemini API key for the Q&A and quiz functions.
- Internet access when Gemini-backed features are used.

The API key must not be placed in the documentation or committed to source control. The project reads GEMINI_API_KEY from the environment; GEMINI_MODEL is optional and defaults to gemini-3.6-flash in the current code.

8. System Architecture

The architecture follows a thin web-client/API pattern. The browser collects input and sends JSON POST requests to FastAPI. `main.py` imports the five module functions and routes requests to them. Q&A and quiz generation use the Gemini client; explanation, summary, and learning recommendations currently execute local Python logic.

Figure 1. System Architecture of EduGenie

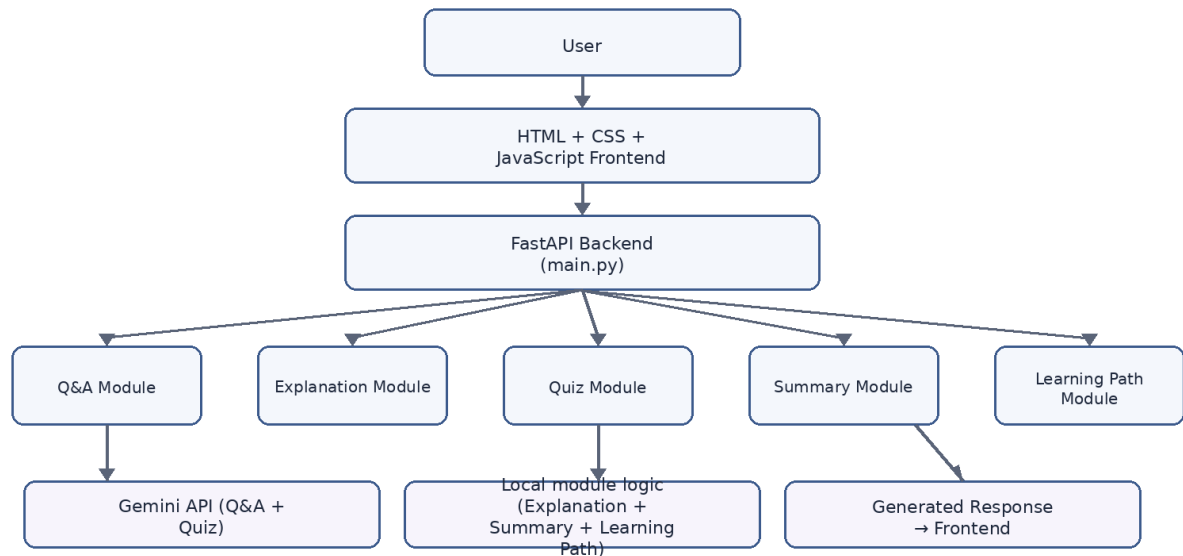


Figure 1. System Architecture of EduGenie

This diagram intentionally differs from the older reference PDF where necessary: the current source does not show a Gemini fallback inside `explanation_module.py`, and the summary and learning-path modules are local functions.

9. Request Flow

The frontend uses the selected module to construct an endpoint and request body. It then sends a POST request, reads the response as JSON when possible, and renders either a normal response or the interactive quiz interface.

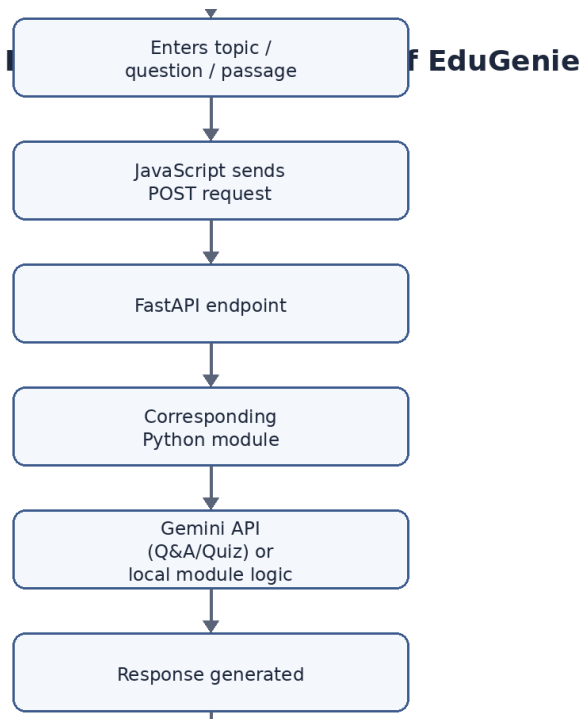


Figure 2. Request Flow of EduGenie

10. Project Folder Architecture

The repository contains the FastAPI application, individual Python modules, frontend template and static assets, configuration/dependency files, and a screenshots folder. The runtime frontend is served from `templates/index.html` and the static mount points to the static directory.

Figure 3. Project Folder Architecture

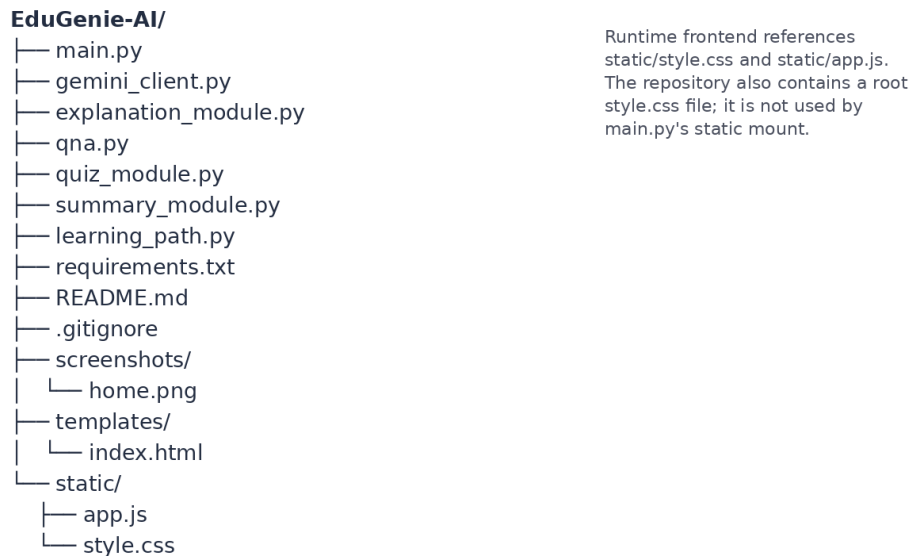


Figure 3. Project Folder Architecture

11. Milestone 1 – Model Selection and Architecture

11.1 Model Selection

The current implementation uses Google Gemini for the two modules that need generative responses: Q&A and quiz generation. The shared `gemini_client.py` reads `GEMINI_API_KEY` and `GEMINI_MODEL` from the environment and exposes `generate_text()`. Q&A also contains its own client/retry handling. The remaining three modules are local.

The original reference PDF described Gemini 1.5 Pro and LaMini-Flan-T5. The current repository instead defaults to `gemini-3.6-flash` in `gemini_client.py` and `qna.py`, while the visible explanation implementation is template-based. This documentation therefore records the implementation that can be verified in source.

11.2 Folder Architecture

The module-oriented folder structure separates routing, AI client configuration, educational logic, and frontend assets. This is suitable for a student project because each module has a clear responsibility and can be tested or modified independently.

11.3 System Architecture Diagram

Figure 1. System Architecture of EduGenie

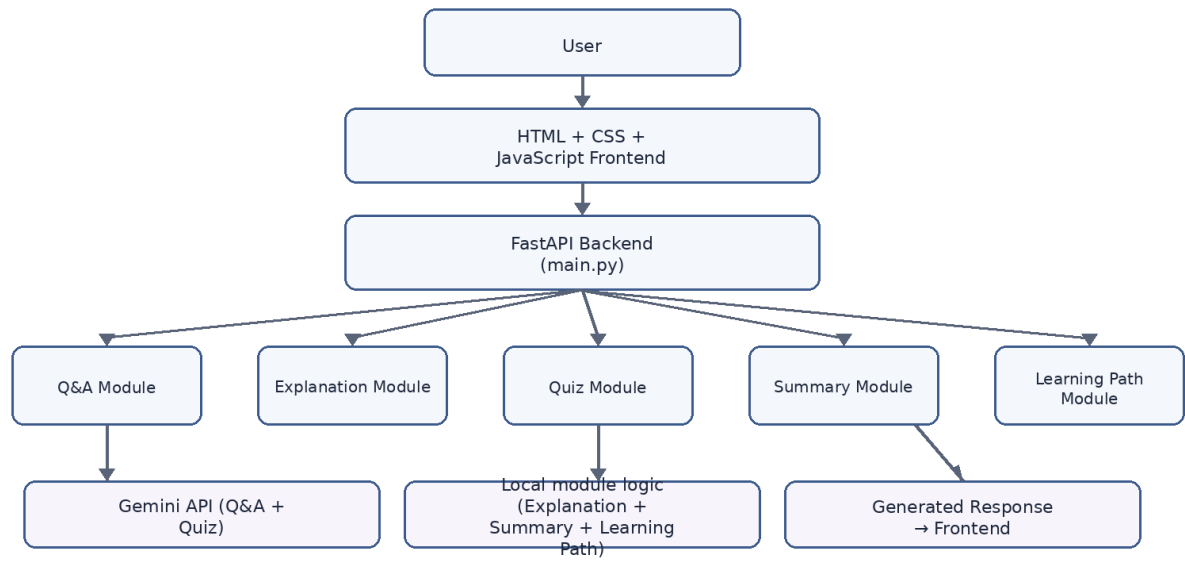


Figure 1. System Architecture of EduGenie

12. Milestone 2 – Backend Module Development

12.1 Explanation Module

The current explanation module is intentionally small. It accepts a topic string and returns a fixed educational template containing the topic, a simple definition-oriented explanation, and suggestions to use definitions, examples, and practice questions.

Figure 4. Explanation Module Logic

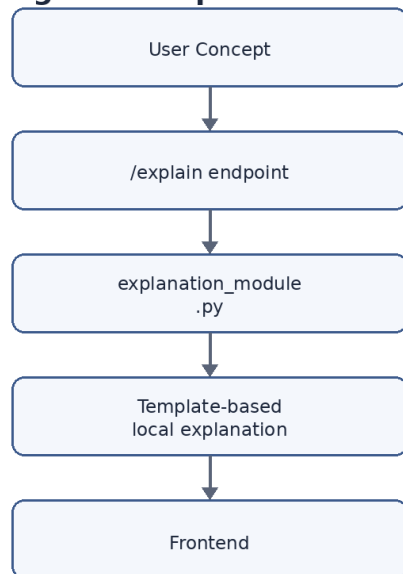


Figure 4. Explanation Module Logic

Program 4. Explanation Module

```
def explain_topic(topic: str) -> str:
    return (
        f"{topic} is an important concept in learning. "
        "A simple explanation is: it is a subject or idea "
        "that helps build understanding. "
        "To learn it well, break it into definitions, "
        "examples, and practice questions."
    )
```

Explanation: The function does not call Gemini or a transformer model in the current repository. It formats the topic into a reusable local explanation template.

12.2 Q&A Module

Q&A is the main free-form generative module. `answer_question()` reads the Gemini credentials from the environment, creates a Gemini client, sends the user's question as the model contents, extracts generated text, and retries temporary 503/UNAVAILABLE failures up to three attempts.

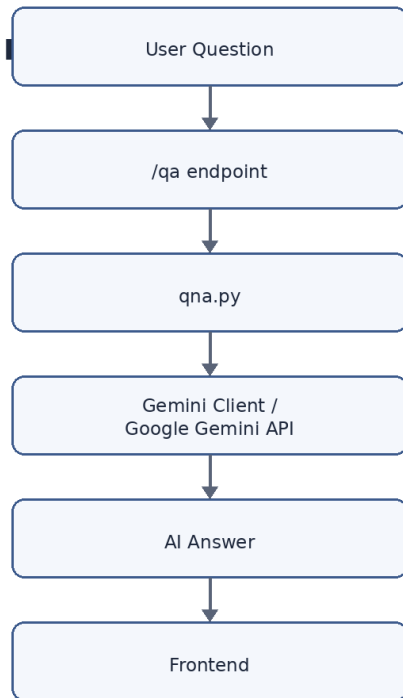


Figure 5. Q&A Module

Program 3. Q&A Module

```
def answer_question(question: str) -> str:
    api_key = os.getenv("GEMINI_API_KEY")
    model = os.getenv("GEMINI_MODEL", "gemini-3.6-flash")

    if not api_key or not api_key.strip():
        raise ValueError("Missing GEMINI_API_KEY in .env.")

    for attempt in range(3):
        try:
            from google import genai
            client = genai.Client(api_key=api_key)
            response = client.models.generate_content(
                model=model,
                contents=question,
            )
            extracted = _extract_generated_text(response)
            if extracted:
                return extracted
            raise ValueError("Gemini returned an empty response.")
        except Exception as exc:
```

```
... # existing retry/error handling
```

Explanation: The documented excerpt shows the actual question-to-Gemini path while eliding the existing response extraction and retry block for readability.

12.3 Quiz Module

The quiz module is the most structured AI component. It asks Gemini for exactly three questions, four options per question, one correct answer, and a short explanation. The response is cleaned, parsed as JSON, and validated before being returned to FastAPI.

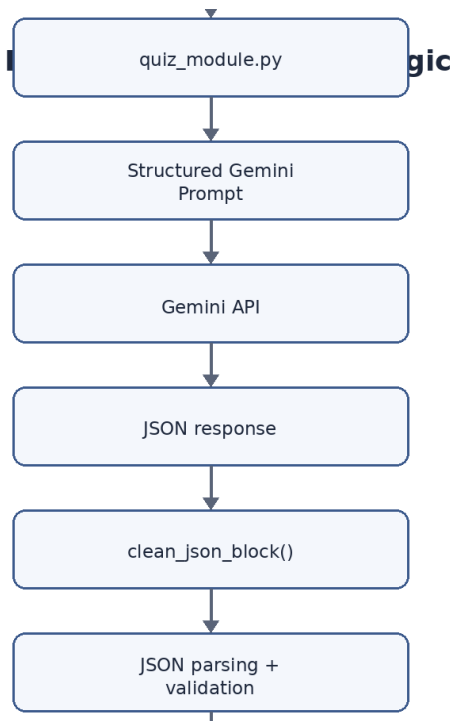


Figure 6. Quiz Module Logic

Program 5. Quiz Generation Module

```
import json
import re
from gemini_client import generate_text

def clean_json_block(text: str) -> str:
    text = text.strip()
    text = re.sub(r"^(?:json)?s*", "", text, flags=re.IGNORECASE)
    text = re.sub(r"s*", "", text)
    start, end = text.find("{"), text.rfind("}")
    if start != -1 and end != -1:
        text = text[start:end + 1]
    return text.strip()
```

```
def generate_quiz(topic: str):
    prompt = f"""Create exactly 3 questions about {topic}.
    Each question must have exactly 4 options (A-D),
    one correct answer, and a short explanation.
    Return ONLY valid JSON."""
    raw_response = generate_text(prompt)
    quiz = json.loads(clean_json_block(raw_response))

    if "questions" not in quiz or len(quiz["questions"]) != 3:
        raise ValueError("Quiz must contain exactly 3 questions.")

    for question in quiz["questions"]:
        for field in ["question", "options", "answer", "explanation"]:
            if field not in question:
                raise ValueError(f'Missing '{field}'')
        if len(question["options"]) != 4:
            raise ValueError("Each question must contain exactly 4 options.")
        if question["answer"] not in question["options"]:
            raise ValueError("Answer must be one of the options.")
    return quiz
```

Explanation: `clean_json_block()` removes optional Markdown fences and isolates the JSON object. `generate_quiz()` then parses and validates the expected three-question structure.

12.4 Summary Module

The current summary module is local rather than Gemini-backed. It splits the input on periods, removes empty fragments, and returns the first three fragments. If no text is supplied, it returns a fixed message.

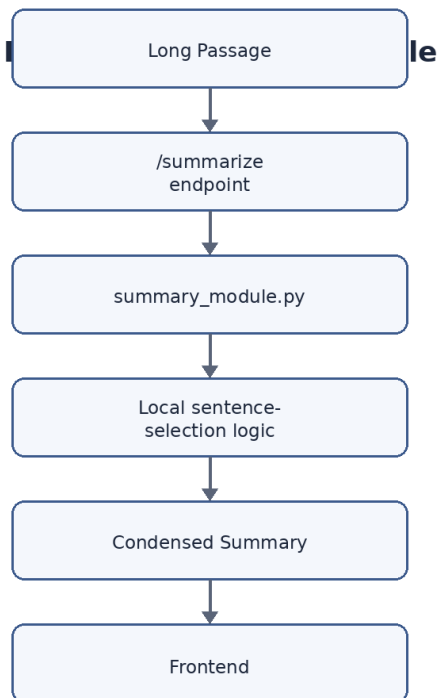


Figure 7. Summary Module

Program 6. Summary Module

```
def summarize_text(text: str) -> str:
    sentences = [part.strip() for part in text.split(".") if part.strip()]
    if not sentences:
        return "No text provided to summarize."
    return " ".join(sentences[:3]) + ("..." if len(sentences) > 3 else "")
```

Explanation: The function implements a lightweight extractive-style condensation rather than generative summarization.

12.5 Learning Path Module

The learning-path module accepts a topic and a level. Beginner and intermediate levels use the base recommendation list in the current implementation; advanced uses a different list containing more challenging activities.

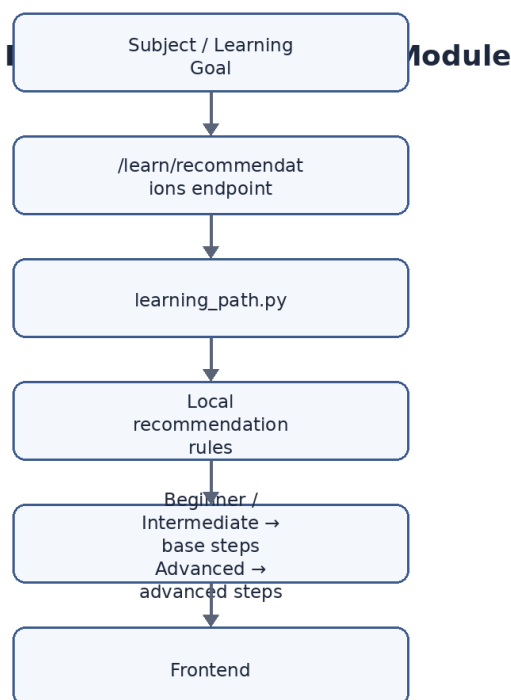


Figure 8. Learning Path Module

Program 7. Learning Path Module

```
def get_learning_recommendations(
    topic: str, level: str = "beginner"
) -> list[str]:
    base = [
        f"Learn the basic definition of {topic}.",
        f"Study one real-world example of {topic}.",
```

```

    f"Practice 5 short questions about {topic}.",
    f"Review the topic and write a summary in your own words.",
]

if level.lower() == "advanced":
    return [
        f"Analyze complex cases related to {topic}.",
        f"Compare {topic} with a similar concept.",
        f"Build a small project or challenge around {topic}.",
        f"Explain {topic} to someone else using examples.",
    ]
return base

```

Explanation: The visible source uses local rules. It does not call Gemini, and it does not automatically generate a timed beginner-to-advanced roadmap.

12.6 FastAPI Backend

main.py creates the FastAPI application, configures Jinja2 templates and the /static mount, imports the five module functions, defines validated request models, and exposes the application endpoints.

Figure 9. FastAPI Endpoints

GET	/
POST	/qa
POST	/explain
POST	/quiz
POST	/summarize
POST	/learn/recommendations
GET	/health

Figure 9. FastAPI Endpoints

Program 1. FastAPI Backend

```

from pathlib import Path
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse, JSONResponse

```

```

from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles

from qna import answer_question
from explanation_module import explain_topic
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

BASE_DIR = Path(__file__).resolve().parent

app = FastAPI(
    title="EduGenie",
    description="Google Gemini Powered Learning Assistant",
    version="1.0.0",
)

templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))
app.mount("/static", StaticFiles(directory=str(BASE_DIR / "static")), name="static")

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        request=request, name="index.html", context={"title": "EduGenie"}
    )

@app.post("/qa")
async def qa(request: QuestionRequest):
    result = answer_question(request.question)
    return {"success": True, "task": "qa", "result": result}

@app.post("/explain")
async def explain(request: ExplainRequest):
    result = explain_topic(request.topic)
    return {"success": True, "task": "explain", "result": result}

@app.post("/quiz")
async def quiz(request: TextRequest):
    result = generate_quiz(request.text)
    return {"success": True, "task": "quiz", "result": result.model_dump() if hasattr(result, "model_dump") else result}

@app.post("/summarize")
async def summarize(request: TextRequest):
    result = summarize_text(request.text)
    return {"success": True, "task": "summarize", "result": result}

@app.post("/learn/recommendations")
async def learning_recommendations(request: LearningPathRequest):
    result = get_learning_recommendations(request.topic, request.level or "beginner")
    return {"success": True, "task": "learning_path", "result": result}

```

```
@app.get("/health")
async def health():
    return {"status": "ok", "application": "EduGenie", "version": "1.0.0"}
```

Explanation: The excerpt covers application initialization, template/static configuration, the root route, the five POST module routes, and the health check. Request-model class definitions and repeated exception wrappers are shortened.

13. Milestone 3 – Frontend Development

13.1 Frontend Structure

The frontend is composed of templates/index.html plus static/app.js and static/style.css. index.html presents a module selector, module-specific input areas, a Generate button, and a response area. app.js controls which input is visible, builds the request, sends the POST call, and renders the response. Quiz responses are rendered into radio-button questions with client-side answer checking.

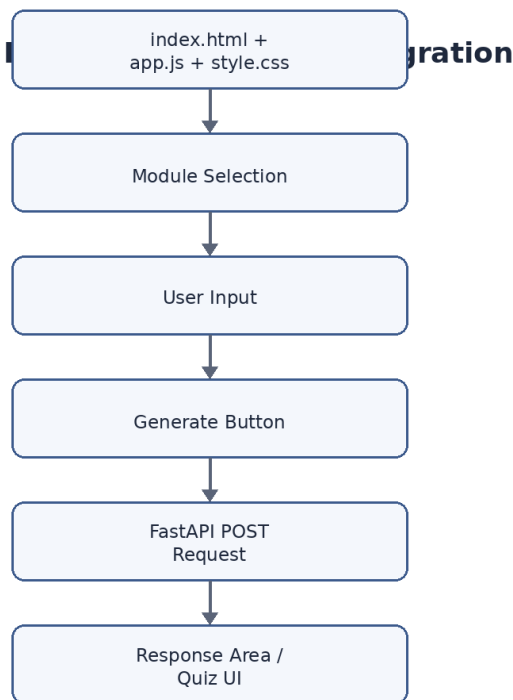


Figure 10. Frontend Integration

Program 8. Frontend HTML

```
<select id="module-select">
  <option value="qa">Q&A - Ask a Question</option>
  <option value="explain">Explain - Simplify a Topic</option>
  <option value="quiz">Quiz - Generate MCQ Quiz</option>
  <option value="summarize">Summary - Summarize Text</option>
  <option value="learning">Recommend Path - Learning Roadmap</option>
</select>

<textarea id="qa-input" rows="6"></textarea>
<textarea id="explain-input" rows="6"></textarea>
<textarea id="quiz-input" rows="6"></textarea>
<textarea id="summary-input" rows="8"></textarea>
<textarea id="learning-input" rows="6"></textarea>
<select id="learning-level">...</select>

<button id="generate-button" type="button">Generate</button>
```

```

<section class="response-card">
  <h2>EduGenie Response</h2>
  <div id="response-output"></div>
</section>

<script src="{{ url_for('static', path='app.js') }}?v=2"></script>

```

Explanation: The important structure includes the module selector, module-specific textareas, learning level selector, Generate button, response container, and app.js inclusion.

Program 9. Frontend JavaScript

```

moduleSelect.addEventListener("change", () => {
  showModule(moduleSelect.value);
});

function getRequestData(module) {
  if (module === "qa") {
    return { endpoint: "/qa",
      data: { question: document.getElementById("qa-input").value.trim() } };
  }
  if (module === "explain") {
    return { endpoint: "/explain",
      data: { topic: document.getElementById("explain-input").value.trim() } };
  }
  if (module === "quiz") {
    return { endpoint: "/quiz",
      data: { text: document.getElementById("quiz-input").value.trim() } };
  }
  ...
}

const response = await fetch(request.endpoint, {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify(request.data)
});

const data = await response.json();

if (module === "quiz") {
  renderQuiz(data);
} else {
  renderNormalResponse(data);
}

```

Explanation: The JavaScript maps module selections to FastAPI endpoints, sends JSON POST requests, and chooses renderQuiz() for quizzes or renderNormalResponse() for other results.

Program 10. Frontend CSS

```
.quiz-score {  
  margin-top: 18px;  
  padding: 18px;  
  border-radius: 12px;  
  background: #edf2ff;  
  border: 1px solid #d8e0ff;  
  text-align: center;  
}  
  
@media (max-width: 700px) {  
  .topbar { padding: 16px 20px; }  
  .container { width: min(100% - 20px, 1000px); }  
  .response-output { padding: 16px; }  
  .quiz-question { padding: 15px; }  
}
```

Explanation: The excerpt demonstrates quiz-result styling and a mobile media query. The full stylesheet contains the rest of the interface styling.

14. Milestone 4 – Running and Testing

14.1 Installation

A Windows-oriented local setup matching the evidence screenshots is:

```
python -m venv venv
```

```
.\venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
uvicorn main:app --reload
```

Open the local application at <http://127.0.0.1:8000>.

14.2 Virtual Environment

The virtual environment isolates the project dependencies from the system Python installation. It is particularly useful because the requirements file includes FastAPI, Uvicorn, Google GenAI, Pydantic, testing packages, and optional machine-learning packages.

14.3 Dependency Installation

Dependencies are installed with pip using requirements.txt. The repository's requirements file was used as the source for the dependency list in this document.

14.4 Uvicorn Run Command

```
uvicorn main:app --reload
```

14.5 Local Application

The FastAPI root route serves index.html. The browser then loads the static JavaScript and CSS files through the mounted /static path. The evidence screenshot shows the application running locally at 127.0.0.1:8000.

14.6 Testing Process

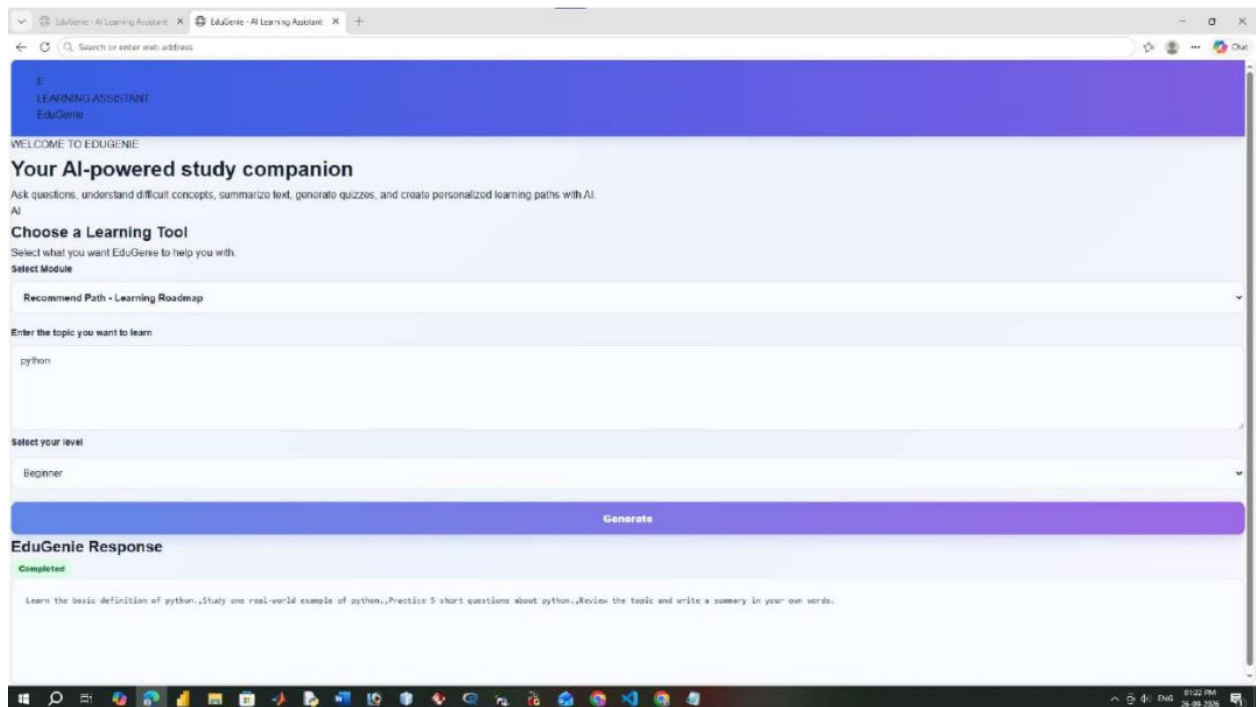
1. Open the local application.
2. Select one learning module.
3. Enter the required topic, question, or text.
4. Click Generate.
5. Observe the Completed or Error status.
6. For quizzes, select answers and use Check Answers to calculate the client-side score.
7. Repeat the process for the other available modules.

15. Project Evidence

Only screenshots present in the uploaded evidence PDF were used below. No new screenshots were fabricated. The uploaded evidence contains nine pages, including a repeated summary screenshot; therefore only distinct evidence is reproduced here.

Application / Learning Path Screenshot

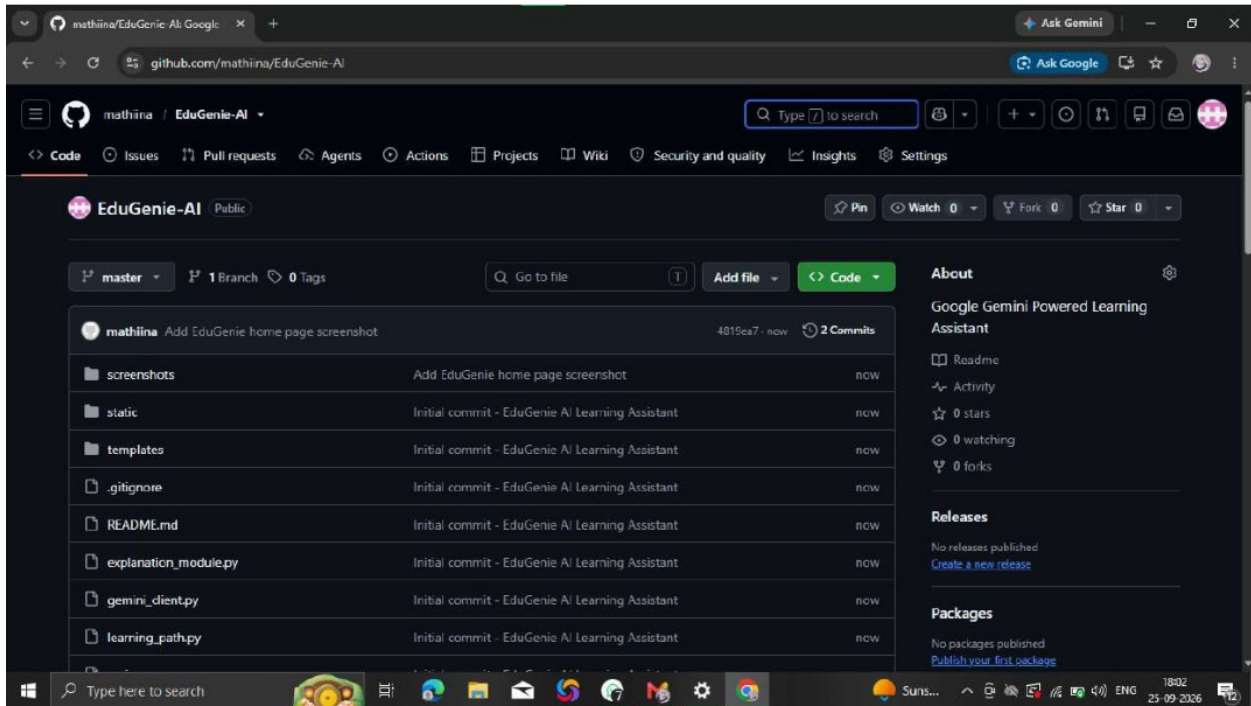
The local EduGenie interface is shown with Recommend Path selected and a Python learning topic.



Evidence 1. Application / Learning Path Screenshot

GitHub Repository Screenshot

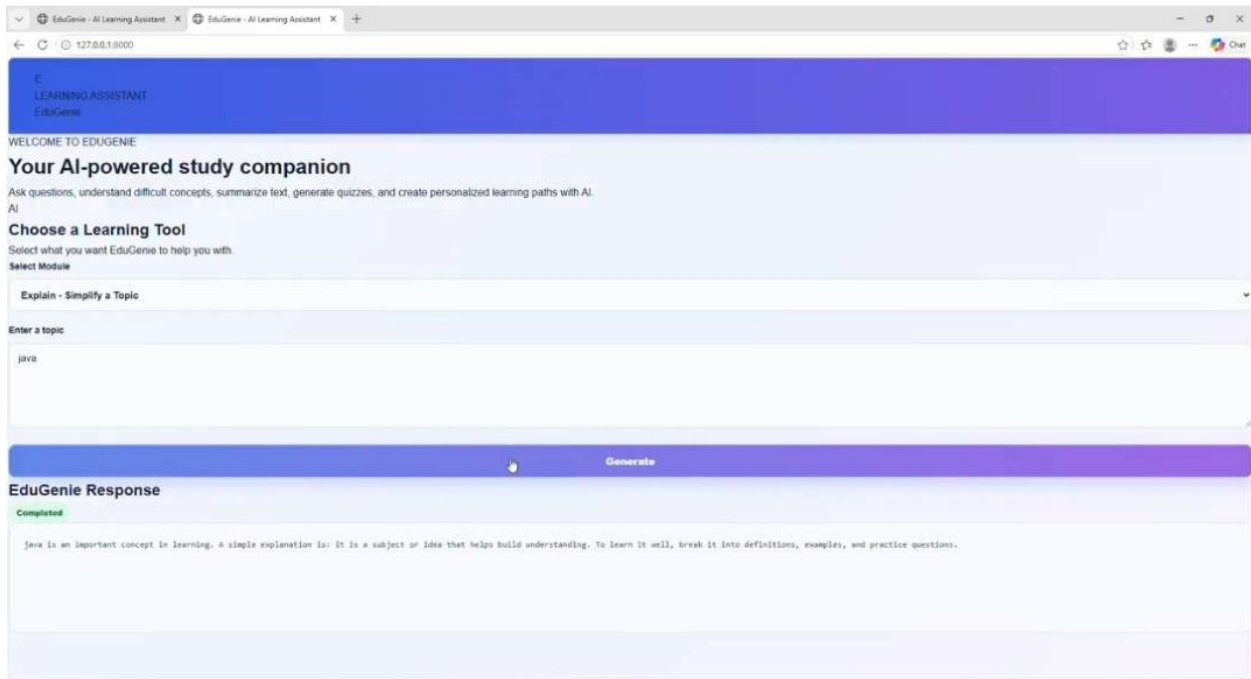
The public EduGenie-AI repository and project files are visible.



Evidence 2. GitHub Repository Screenshot

Simplified Topic / Website Screenshot

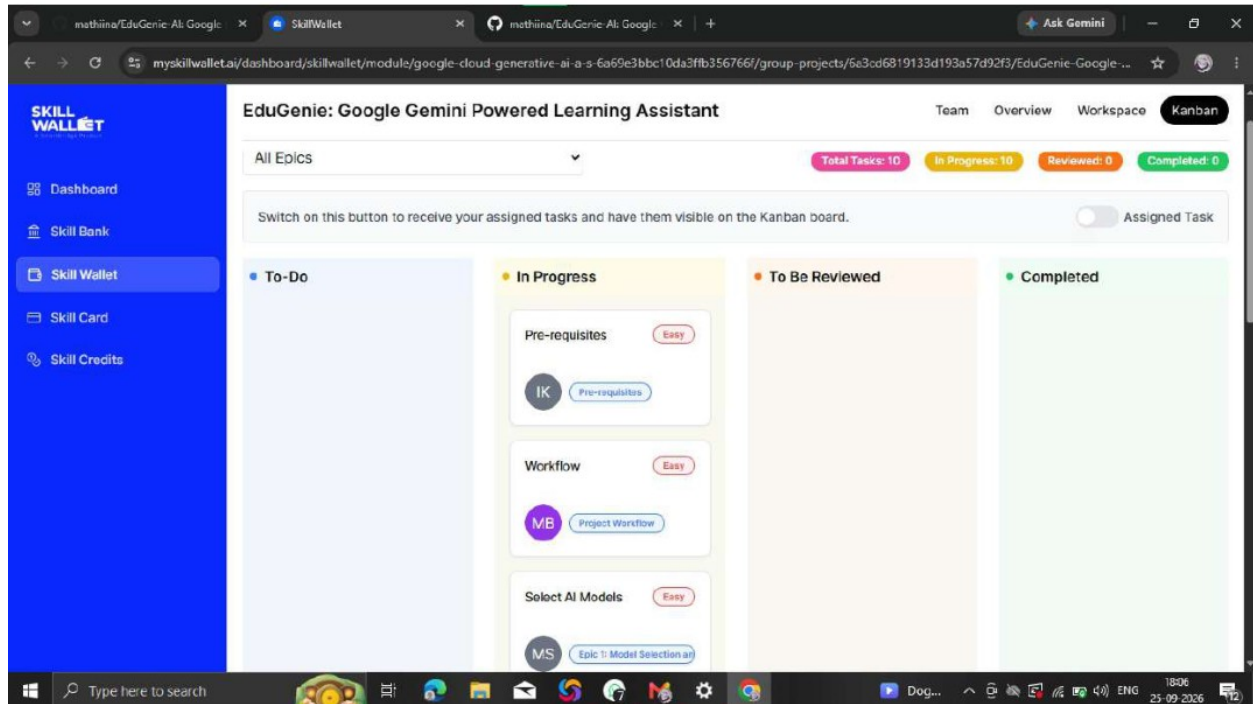
The Explain module is shown with a topic entered and a completed response.



Evidence 3. Simplified Topic / Website Screenshot

TODO / Development Screenshot

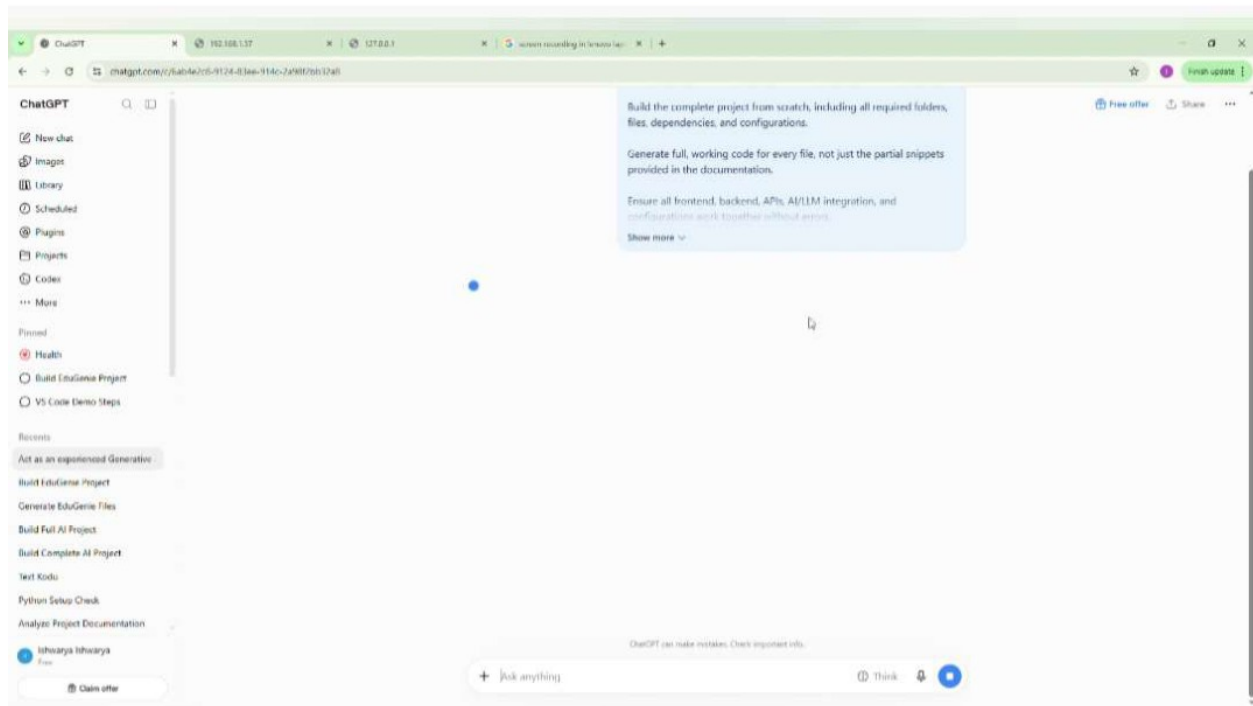
A Skill Wallet/Kanban view is shown with project tasks including model-selection work.



Evidence 4. TODO / Development Screenshot

ChatGPT Prompt Screenshot

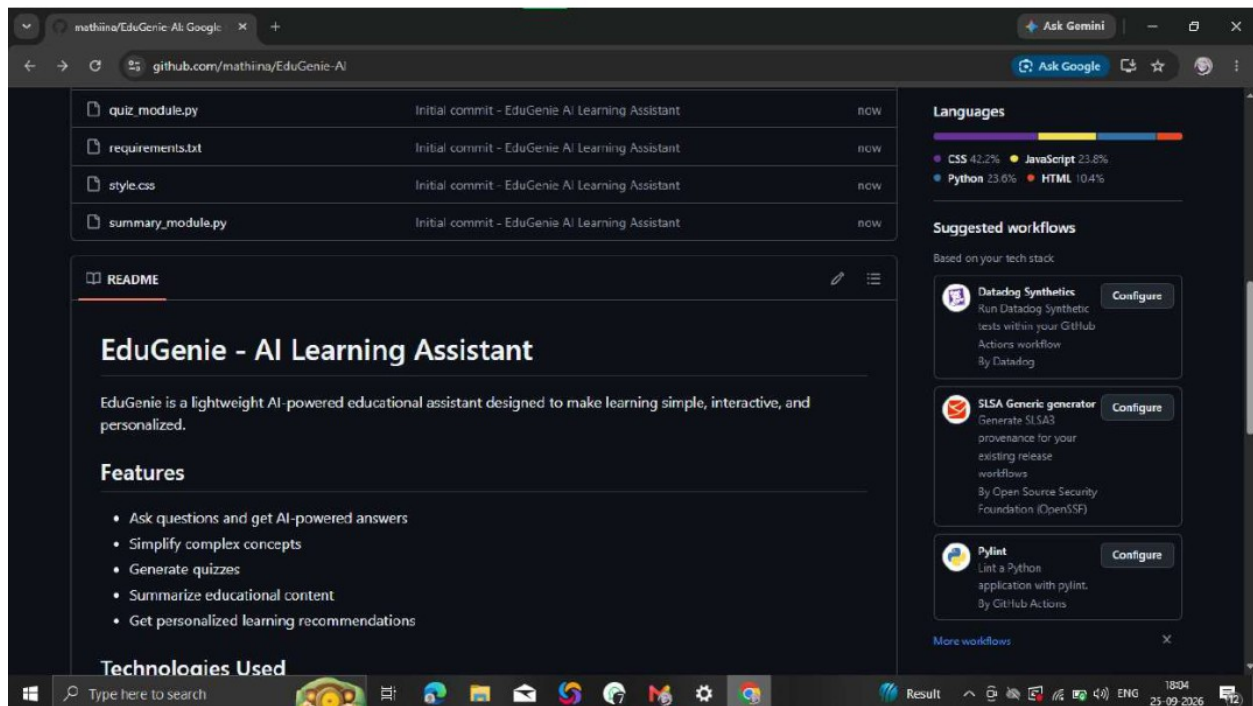
The prompt requests the project to be built with required folders, files, dependencies, and configuration.



Evidence 5. ChatGPT Prompt Screenshot

GitHub README Screenshot

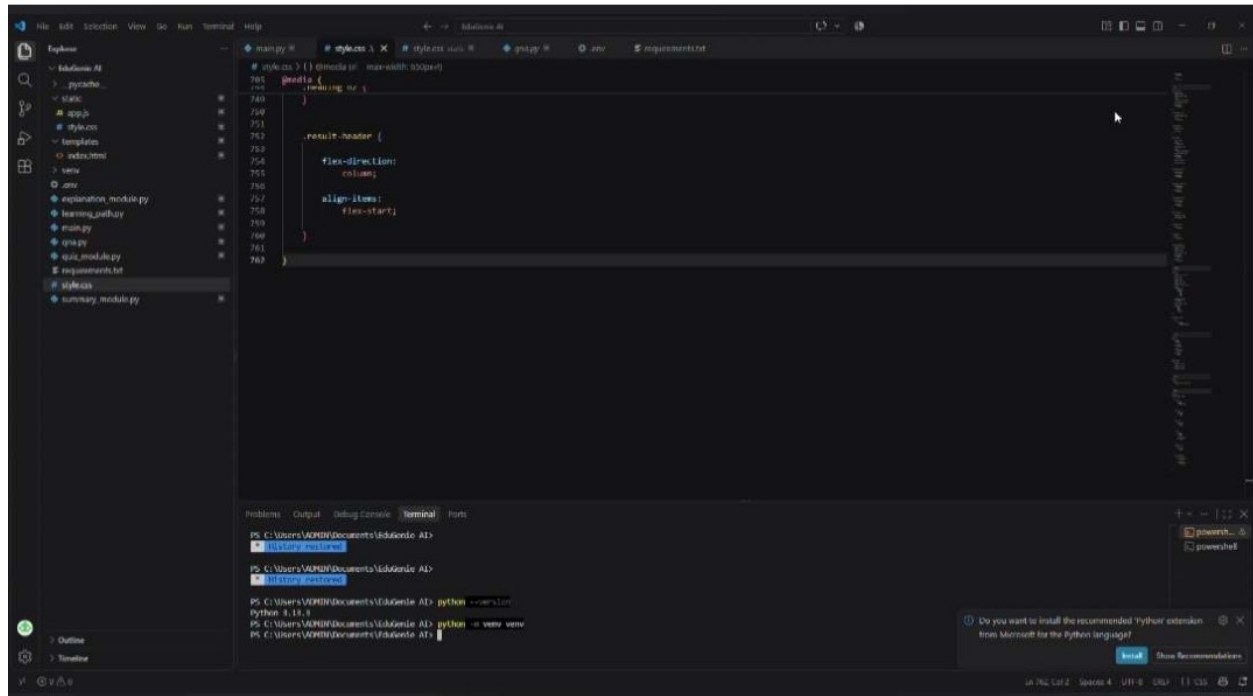
The repository README shows the project description and feature list.



Evidence 6. GitHub README Screenshot

Program / Terminal Screenshot

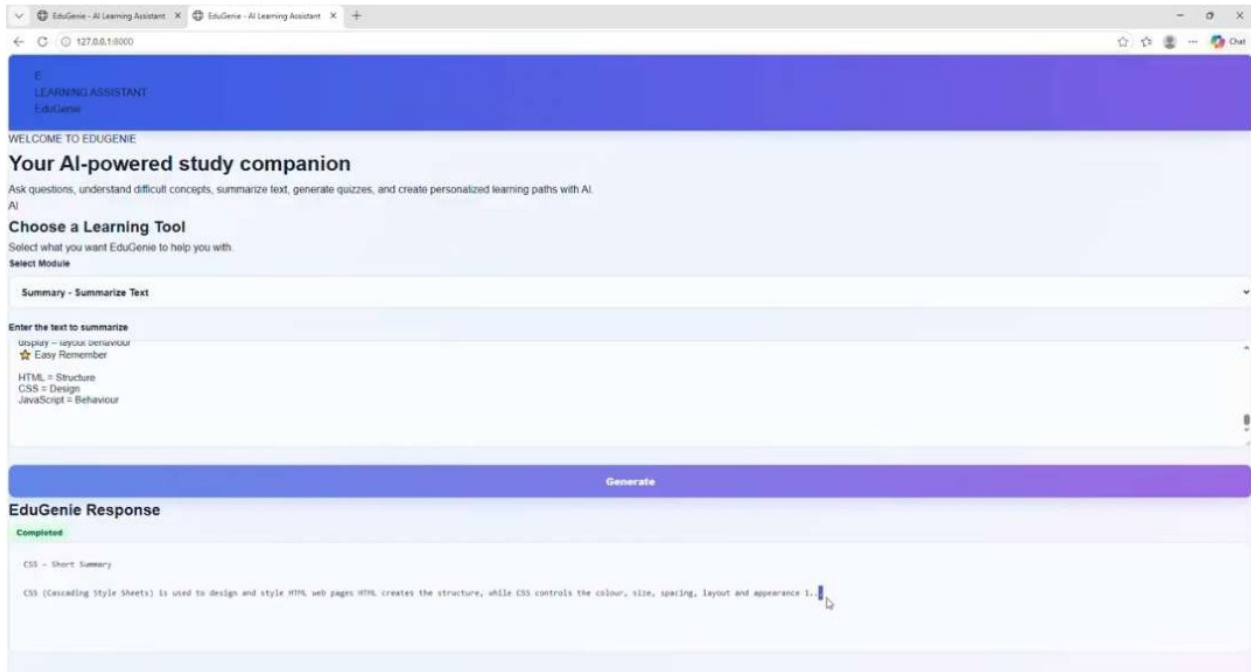
Visual Studio Code shows the project source tree, CSS code, and a terminal/virtual-environment area.



Evidence 7. Program / Terminal Screenshot

Summary Screenshot

The Summary module is shown with input text and a completed local result.



Evidence 8. Summary Screenshot

15.1 Evidence Availability Matrix

Requested Evidence	Status in Uploaded Evidence	Documentation Treatment
Terminal	Partly visible in Program screenshot	Use the combined Program/Terminal evidence; no separate terminal screenshot invented.
App	Available	Application screenshot included.
ChatGPT Prompt	Available	Prompt screenshot included.
Website	Available	Application/website screenshots included.
GitHub	Available	Repository screenshots included.
TODO	Available	Development/Kanban screenshot included.
Program	Available	VS Code source screenshot included.
Quiz	No distinct screenshot in uploaded evidence PDF	No fabricated quiz screenshot.
Simplified Topic	Available	Explain screenshot included.
Learning Path	Available	Recommend Path screenshot

included.

Question & Answer

No distinct screenshot in uploaded
evidence PDF

No fabricated Q&A screenshot.

16. Git and GitHub Workflow

The repository evidence shows the project stored in GitHub. A standard workflow for this project is to initialize the repository, stage project files, create a commit, connect the local repository to its remote, and push the commit.

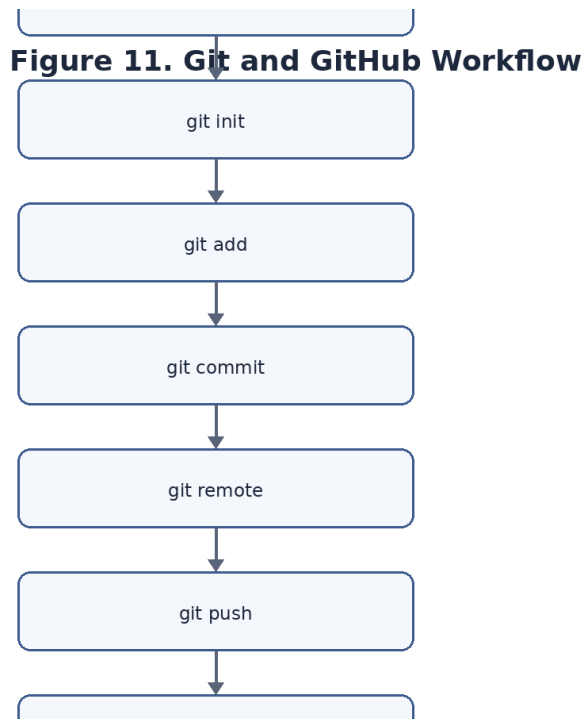


Figure 11. Git and GitHub Workflow

`git init`

`git add .`

`git commit -m "Initial EduGenie project"`

`git remote add origin <repository-url>`

`git push -u origin master`

17. Repository Safety

- Never commit `.env` files containing a real Gemini API key.
- Keep `GEMINI_API_KEY` in the environment rather than in Python source.
- Use `.gitignore` to prevent local secrets and generated files from being committed.
- Before sharing the repository, search the history and current files for accidental credentials.
- The documentation intentionally omits all real API keys and `.env` contents.

The current `gemini_client.py` raises an error if `GEMINI_API_KEY` is missing, which makes the configuration requirement explicit.

18. Advantages

- Simple modular architecture.
- FastAPI provides clear REST endpoints and automatic request validation through Pydantic.
- Q&A and quiz generation benefit from Gemini's generative capabilities.
- Quiz output is explicitly cleaned, parsed, and validated before reaching the frontend.
- The frontend keeps all modules inside one consistent user interface.
- The project is lightweight enough for local development and student demonstration.

19. Limitations

- Explanation is currently a fixed local template rather than a true topic-aware model.
- Summary currently returns at most the first three sentence fragments and is not semantic summarization.
- Learning-path recommendations are fixed rule-based lists rather than generated personalized roadmaps.
- Q&A and quiz generation depend on Gemini configuration and network/API availability.
- The frontend performs quiz scoring in the browser; it is not a server-side assessment system.
- The README and current source differ on the role of LaMini-Flan-T5, so model documentation should be synchronized with future code changes.

20. Future Enhancements

- Replace the fixed explanation template with a real local explanation model or a controlled Gemini prompt.
- Upgrade summary generation to semantic summarization with configurable length.
- Generate structured learning paths using Gemini while validating the returned structure.
- Add persistent user progress and history.
- Add backend-side quiz scoring and result storage.
- Add automated tests for each endpoint and module.
- Add API documentation examples and request/response schemas.
- Add authentication and rate limiting if the application is deployed publicly.

21. Conclusion

EduGenie demonstrates how a compact FastAPI application can combine browser-based interaction with modular educational functions. The project provides Q&A, explanation, quiz, summary, and learning-path operations through a common frontend and endpoint layer. The current implementation is intentionally lightweight: Gemini is used for Q&A and quiz generation, while the remaining modules use local Python logic. This distinction is important for accurate technical documentation.

The project also demonstrates practical software-development activities including environment configuration, REST API design, frontend integration, JSON validation, local execution with Uvicorn, Git version control, and GitHub-based repository management. Its modular structure leaves a clear path for replacing the current local heuristics with richer educational models in later versions.

22. References

8. Supplied reference document: “EduGenie: Google Gemini Powered Learning Assistant” (uploaded PDF, 18 pages).
9. EduGenie-AI GitHub repository: <https://github.com/mathiina/EduGenie-AI>
10. FastAPI documentation: <https://fastapi.tiangolo.com/>
11. Google Gemini / Google GenAI documentation: <https://ai.google.dev/>
12. Jinja2 documentation: <https://jinja.palletsprojects.com/>
13. Uvicorn documentation: <https://www.uvicorn.org/>

23. Submission Details

THANK YOU